

4.2 Context-Free Grammars

Grammars were introduced in Section 2.2 to systematically describe the syntax of programming language constructs like expressions and statements. Using a syntactic variable *stmt* to denote statements and variable *expr* to denote expressions, the production

$$stmt \rightarrow \text{if } (expr) \text{ } stmt \text{ else } stmt \quad (4.4)$$

specifies the structure of this form of conditional statement. Other productions then define precisely what an *expr* is and what else a *stmt* can be.

This section reviews the definition of a context-free grammar and introduces terminology for talking about parsing. In particular, the notion of derivations is very helpful for discussing the order in which productions are applied during parsing.

4.2.1 The Formal Definition of a Context-Free Grammar

From Section 2.2, a context-free grammar (grammar for short) consists of terminals, nonterminals, a start symbol, and productions.

1. **Terminals** are the basic symbols from which strings are formed. The term “token name” is a synonym for “terminal” and frequently we will use the word “token” for terminal when it is clear that we are talking about just the token name. We assume that the terminals are the first components of the tokens output by the lexical analyzer. In (4.4), the terminals are the keywords **if** and **else** and the symbols “(” and “).”
2. **Nonterminals** are syntactic variables that denote sets of strings. In (4.4), *stmt* and *expr* are nonterminals. The sets of strings denoted by nonterminals help define the language generated by the grammar. Nonterminals impose a hierarchical structure on the language that is key to syntax analysis and translation.
3. In a grammar, one nonterminal is distinguished as the *start symbol*, and the set of strings it denotes is the language generated by the grammar. Conventionally, the productions for the start symbol are listed first.
4. The productions of a grammar specify the manner in which the terminals and nonterminals can be combined to form strings. Each *production* consists of:
 - (a) A nonterminal called the *head* or *left side* of the production; this production defines some of the strings denoted by the head.
 - (b) The symbol $\rightarrow$. Sometimes $:=$ has been used in place of the arrow.
 - (c) A *body* or *right side* consisting of zero or more terminals and nonterminals. The components of the body describe one way in which strings of the nonterminal at the head can be constructed.

Example 4.5: The grammar in Fig. 4.2 defines simple arithmetic expressions. In this grammar, the terminal symbols are

id + - * / ()

The nonterminal symbols are *expression*, *term* and *factor*, and *expression* is the start symbol $\square$

<i>expression</i>	$\rightarrow$	<i>expression</i> + <i>term</i>
<i>expression</i>	$\rightarrow$	<i>expression</i> - <i>term</i>
<i>expression</i>	$\rightarrow$	<i>term</i>
<i>term</i>	$\rightarrow$	<i>term</i> * <i>factor</i>
<i>term</i>	$\rightarrow$	<i>term</i> / <i>factor</i>
<i>term</i>	$\rightarrow$	<i>factor</i>
<i>factor</i>	$\rightarrow$	(<i>expression</i>)
<i>factor</i>	$\rightarrow$	id

Figure 4.2: Grammar for simple arithmetic expressions

4.2.2 Notational Conventions

To avoid always having to state that “these are the terminals,” “these are the nonterminals,” and so on, the following notational conventions for grammars will be used throughout the remainder of this book.

1. These symbols are terminals:

- Lowercase letters early in the alphabet, such as *a*, *b*, *c*.
- Operator symbols such as +, *, and so on.
- Punctuation symbols such as parentheses, comma, and so on.
- The digits 0, 1, ..., 9.
- Boldface strings such as **id** or **if**, each of which represents a single terminal symbol.

2. These symbols are nonterminals:

- Uppercase letters early in the alphabet, such as *A*, *B*, *C*.
- The letter *S*, which, when it appears, is usually the start symbol.
- Lowercase, italic names such as *expr* or *stmt*.
- When discussing programming constructs, uppercase letters may be used to represent nonterminals for the constructs. For example, nonterminals for expressions, terms, and factors are often represented by *E*, *T*, and *F*, respectively.

3. Uppercase letters late in the alphabet, such as X, Y, Z , represent *grammar symbols*; that is, either nonterminals or terminals.
4. Lowercase letters late in the alphabet, chiefly $u, v, \dots, z$, represent (possibly empty) strings of terminals.
5. Lowercase Greek letters, α, β, γ for example, represent (possibly empty) strings of grammar symbols. Thus, a generic production can be written as $A \rightarrow \alpha$, where A is the head and α the body.
6. A set of productions $A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_k$ with a common head A (call them *A-productions*), may be written $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$. Call $\alpha_1, \alpha_2, \dots, \alpha_k$ the *alternatives* for A .
7. Unless stated otherwise, the head of the first production is the start symbol.

Example 4.6: Using these conventions, the grammar of Example 4.5 can be rewritten concisely as

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

The notational conventions tell us that E, T , and F are nonterminals, with E the start symbol. The remaining symbols are terminals. $\square$

4.2.3 Derivations

The construction of a parse tree can be made precise by taking a derivational view, in which productions are treated as rewriting rules. Beginning with the start symbol, each rewriting step replaces a nonterminal by the body of one of its productions. This derivational view corresponds to the top-down construction of a parse tree, but the precision afforded by derivations will be especially helpful when bottom-up parsing is discussed. As we shall see, bottom-up parsing is related to a class of derivations known as “rightmost” derivations, in which the rightmost nonterminal is rewritten at each step.

For example, consider the following grammar, with a single nonterminal E , which adds a production $E \rightarrow - E$ to the grammar (4.3):

$$E \rightarrow E + E \mid E * E \mid - E \mid (E) \mid \text{id} \quad (4.7)$$

The production $E \rightarrow - E$ signifies that if E denotes an expression, then $- E$ must also denote an expression. The replacement of a single E by $- E$ will be described by writing

$$E \Rightarrow -E$$

which is read, “ E derives $-E$.” The production $E \rightarrow (E)$ can be applied to replace any instance of E in any string of grammar symbols by (E) , e.g., $E * E \Rightarrow (E) * E$ or $E * E \Rightarrow E * (E)$. We can take a single E and repeatedly apply productions in any order to get a sequence of replacements. For example,

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(\text{id})$$

We call such a sequence of replacements a *derivation* of $-(\text{id})$ from E . This derivation provides a proof that the string $-(\text{id})$ is one particular instance of an expression.

For a general definition of derivation, consider a nonterminal A in the middle of a sequence of grammar symbols, as in $\alpha A \beta$, where α and β are arbitrary strings of grammar symbols. Suppose $A \rightarrow \gamma$ is a production. Then, we write $\alpha A \beta \Rightarrow \alpha \gamma \beta$. The symbol $\Rightarrow$ means, “derives in one step.” When a sequence of derivation steps $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$ rewrites α_1 to α_n , we say α_1 *derives* α_n . Often, we wish to say, “derives in zero or more steps.” For this purpose, we can use the symbol $\Rightarrow^*$. Thus,

1. $\alpha \Rightarrow^* \alpha$, for any string α , and
2. If $\alpha \Rightarrow^* \beta$ and $\beta \Rightarrow \gamma$, then $\alpha \Rightarrow^* \gamma$.

Likewise, $\Rightarrow^+$ means, “derives in one or more steps.”

If $S \Rightarrow^* \alpha$, where S is the start symbol of a grammar G , we say that α is a *sentential form* of G . Note that a sentential form may contain both terminals and nonterminals, and may be empty. A *sentence* of G is a sentential form with no nonterminals. The *language generated by* a grammar is its set of sentences. Thus, a string of terminals w is in $L(G)$, the language generated by G , if and only if w is a sentence of G (or $S \Rightarrow^* w$). A language that can be generated by a grammar is said to be a *context-free language*. If two grammars generate the same language, the grammars are said to be *equivalent*.

The string $-(\text{id} + \text{id})$ is a sentence of grammar (4.7) because there is a derivation

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id}) \quad (4.8)$$

The strings $E, -E, -(E), \dots, -(\text{id} + \text{id})$ are all sentential forms of this grammar. We write $E \Rightarrow^* -(\text{id} + \text{id})$ to indicate that $-(\text{id} + \text{id})$ can be derived from E .

At each step in a derivation, there are two choices to be made. We need to choose which nonterminal to replace, and having made this choice, we must pick a production with that nonterminal as head. For example, the following alternative derivation of $-(\text{id} + \text{id})$ differs from derivation (4.8) in the last two steps:

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \text{id}) \Rightarrow -(\text{id} + \text{id}) \quad (4.9)$$

Each nonterminal is replaced by the same body in the two derivations, but the order of replacements is different.

To understand how parsers work, we shall consider derivations in which the nonterminal to be replaced at each step is chosen as follows:

1. In *leftmost* derivations, the leftmost nonterminal in each sentential is always chosen. If $\alpha \Rightarrow \beta$ is a step in which the leftmost nonterminal in α is replaced, we write $\alpha \xRightarrow{lm} \beta$.
2. In *rightmost* derivations, the rightmost nonterminal is always chosen; we write $\alpha \xRightarrow{rm} \beta$ in this case.

Derivation (4.8) is leftmost, so it can be rewritten as

$$E \xRightarrow{lm} -E \xRightarrow{lm} -(E) \xRightarrow{lm} -(E + E) \xRightarrow{lm} -(\mathbf{id} + E) \xRightarrow{lm} -(\mathbf{id} + \mathbf{id})$$

Note that (4.9) is a rightmost derivation.

Using our notational conventions, every leftmost step can be written as $wA\gamma \xRightarrow{lm} w\delta\gamma$, where w consists of terminals only, $A \rightarrow \delta$ is the production applied, and γ is a string of grammar symbols. To emphasize that α derives β by a leftmost derivation, we write $\alpha \xRightarrow{*}_{lm} \beta$. If $S \xRightarrow{*}_{lm} \alpha$, then we say that α is a *left-sentential form* of the grammar at hand.

Analogous definitions hold for rightmost derivations. Rightmost derivations are sometimes called *canonical* derivations.

4.2.4 Parse Trees and Derivations

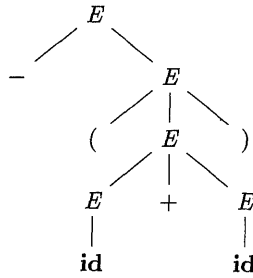
A parse tree is a graphical representation of a derivation that filters out the order in which productions are applied to replace nonterminals. Each interior node of a parse tree represents the application of a production. The interior node is labeled with the nonterminal A in the head of the production; the children of the node are labeled, from left to right, by the symbols in the body of the production by which this A was replaced during the derivation.

For example, the parse tree for $-(\mathbf{id} + \mathbf{id})$ in Fig. 4.3, results from the derivation (4.8) as well as derivation (4.9).

The leaves of a parse tree are labeled by nonterminals or terminals and, read from left to right, constitute a sentential form, called the *yield* or *frontier* of the tree.

To see the relationship between derivations and parse trees, consider any derivation $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$, where α_1 is a single nonterminal A . For each sentential form α_i in the derivation, we can construct a parse tree whose yield is α_i . The process is an induction on i .

BASIS: The tree for $\alpha_1 = A$ is a single node labeled A .

Figure 4.3: Parse tree for $-(\text{id} + \text{id})$

INDUCTION: Suppose we already have constructed a parse tree with yield $\alpha_{i-1} = X_1X_2 \cdots X_k$ (note that according to our notational conventions, each grammar symbol X_i is either a nonterminal or a terminal). Suppose α_i is derived from α_{i-1} by replacing X_j , a nonterminal, by $\beta = Y_1Y_2 \cdots Y_m$. That is, at the i th step of the derivation, production $X_j \rightarrow \beta$ is applied to α_{i-1} to derive $\alpha_i = X_1X_2 \cdots X_{j-1}\beta X_{j+1} \cdots X_k$.

To model this step of the derivation, find the j th leaf from the left in the current parse tree. This leaf is labeled X_j . Give this leaf m children, labeled $Y_1, Y_2, \dots, Y_m$, from the left. As a special case, if $m = 0$, then $\beta = \epsilon$, and we give the j th leaf one child labeled ϵ .

Example 4.10: The sequence of parse trees constructed from the derivation (4.8) is shown in Fig. 4.4. In the first step of the derivation, $E \Rightarrow -E$. To model this step, add two children, labeled $-$ and E , to the root E of the initial tree. The result is the second tree.

In the second step of the derivation, $-E \Rightarrow -(E)$. Consequently, add three children, labeled $($, E , and $)$, to the leaf labeled E of the second tree, to obtain the third tree with yield $-(E)$. Continuing in this fashion we obtain the complete parse tree as the sixth tree. $\square$

Since a parse tree ignores variations in the order in which symbols in sentential forms are replaced, there is a many-to-one relationship between derivations and parse trees. For example, both derivations (4.8) and (4.9), are associated with the same final parse tree of Fig. 4.4.

In what follows, we shall frequently parse by producing a leftmost or a rightmost derivation, since there is a one-to-one relationship between parse trees and either leftmost or rightmost derivations. Both leftmost and rightmost derivations pick a particular order for replacing symbols in sentential forms, so they too filter out variations in the order. It is not hard to show that every parse tree has associated with it a unique leftmost and a unique rightmost derivation.

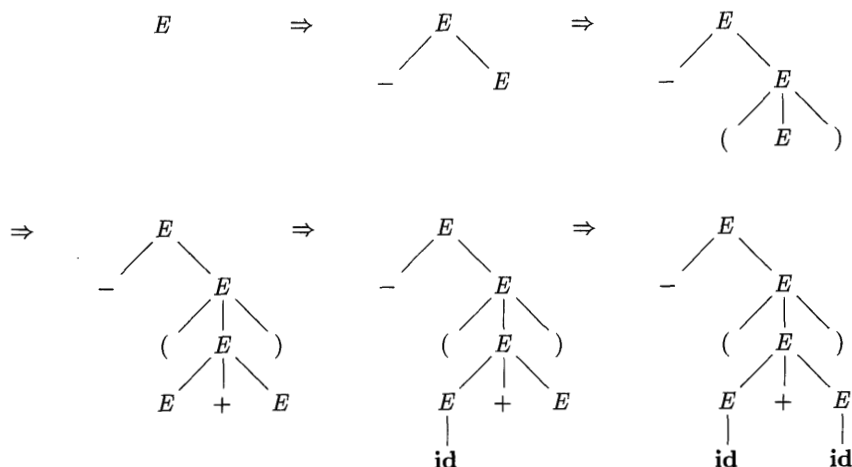


Figure 4.4: Sequence of parse trees for derivation (4.8)

4.2.5 Ambiguity

From Section 2.2.4, a grammar that produces more than one parse tree for some sentence is said to be *ambiguous*. Put another way, an ambiguous grammar is one that produces more than one leftmost derivation or more than one rightmost derivation for the same sentence.

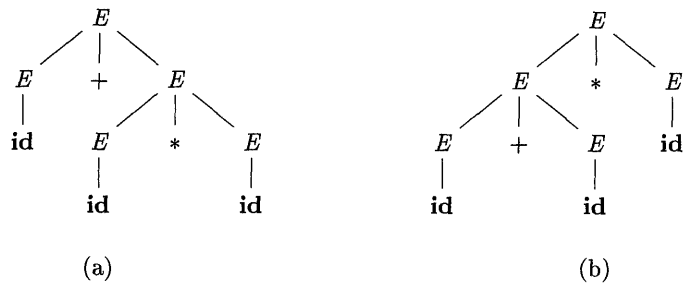
Example 4.11: The arithmetic expression grammar (4.3) permits two distinct leftmost derivations for the sentence $\text{id} + \text{id} * \text{id}$:

$$\begin{array}{ll}
 E \Rightarrow E + E & E \Rightarrow E * E \\
 \Rightarrow \text{id} + E & \Rightarrow E + E * E \\
 \Rightarrow \text{id} + E * E & \Rightarrow \text{id} + E * E \\
 \Rightarrow \text{id} + \text{id} * E & \Rightarrow \text{id} + \text{id} * E \\
 \Rightarrow \text{id} + \text{id} * \text{id} & \Rightarrow \text{id} + \text{id} * \text{id}
 \end{array}$$

The corresponding parse trees appear in Fig. 4.5.

Note that the parse tree of Fig. 4.5(a) reflects the commonly assumed precedence of $+$ and $*$, while the tree of Fig. 4.5(b) does not. That is, it is customary to treat operator $*$ as having higher precedence than $+$, corresponding to the fact that we would normally evaluate an expression like $a + b * c$ as $a + (b * c)$, rather than as $(a + b) * c$. $\square$

For most parsers, it is desirable that the grammar be made unambiguous, for if it is not, we cannot uniquely determine which parse tree to select for a sentence. In other cases, it is convenient to use carefully chosen ambiguous grammars, together with *disambiguating rules* that “throw away” undesirable parse trees, leaving only one tree for each sentence.

Figure 4.5: Two parse trees for **id+id*id**